

# canjob

## Intelligent Candidate Discovery & Ranking

A JD-faithful, CPU-only, fully offline ranking engine that scores 100,000 candidates in ~30 seconds and explains every pick with profile evidence.

### Problem

Data & AI Challenge:  
Intelligent Candidate Discovery

### Team

canjob

### Team Leader

Navpreet Singh

# What we built and why it is different

- ▶ **Solution:** an ensemble ranker that turns a JD into config, then scores every candidate with three independent lenses and fuses them.
- ▶ **Reads profiles, not just keywords:** a deterministic JD-faithful rules model encodes must-haves and the disqualifiers the JD spells out.
- ▶ **Behavior-aware:** the 23 Redrob signals modulate score so unreachable perfect-on-paper profiles sink.
- ▶ **Honest by design:** logically-impossible honeypots are detected and removed; every row carries evidence-based reasoning.

## 3 lenses -> 1 rank

Rules + lexical TF-IDF + MiniLM semantic, fused with Reciprocal Rank Fusion.

## No LLM at rank time

Differentiator vs GPT-per-candidate systems that cannot scale to a 200k pool in production.

## Config-driven

One codebase serves many jobs; each JD is its own versioned config folder.

# Key requirements and the signals that matter

## Extracted from the JD

- ▶ Embeddings & dense/hybrid retrieval; vector DBs (Pinecone, FAISS, Qdrant)
- ▶ Ranking / recsys + evaluation literacy (NDCG, MRR, MAP, A/B)
- ▶ LLM fine-tuning / RAG, production Python at scale
- ▶ Product-company builder profile, 5-9 yrs, founding-team mindset

## Beyond keyword matching

- ▶ **Semantic fit:** MiniLM matches meaning so a recommender-systems builder ranks even without the buzzwords.
- ▶ **Availability:** response rate, recency, open-to-work, interview/offer reliability, recruiter demand.
- ▶ **Trust:** endorsements + skill duration discount lazy keyword stuffing.
- ▶ **Disqualifiers:** off-domain title with stuffed AI skills, framework-only, title-chaser tenures.

# Retrieve, score, rank, fuse

### Rules (deterministic)

JD-faithful score from config: skill facets, experience band, product vs services, location. Minus explicit penalties. x behavioral modifier.

### Lexical (TF-IDF)

Several focused queries (retrieval, ranking/eval, full JD) for keyword recall instead of one blurry mega-query.

### Semantic (MiniLM)

Offline sentence embeddings, precomputed once. Cosine-to-JD captures meaning the other lenses miss.

### Fusion: Reciprocal Rank Fusion (RRF)

Each lens produces a ranking; fused score =  $\text{sum of } \text{weight} / (k + \text{rank})$ . Combining on ranks, not raw scores, means no single lens scale can dominate. Weights live in `jd_facets.json`. Honeypots are excluded; ties break by `candidate_id`; scores are non-increasing.

## Grounded reasoning, no hallucination, suspicious profiles

- ▶ **Evidence-based:** reasoning cites the candidate's own named skills (proficiency + months), employers, and evaluation terms found in their text.
- ▶ **Mapped to the JD:** each clause connects a concrete fact to a specific JD requirement, not generic praise.
- ▶ **Honest concerns:** gaps (services-only career, framework-only, long notice) are stated, and tone matches the rank.
- ▶ **No hallucination:** every claim is pulled from the profile; nothing is invented, so Stage-4 checks pass.

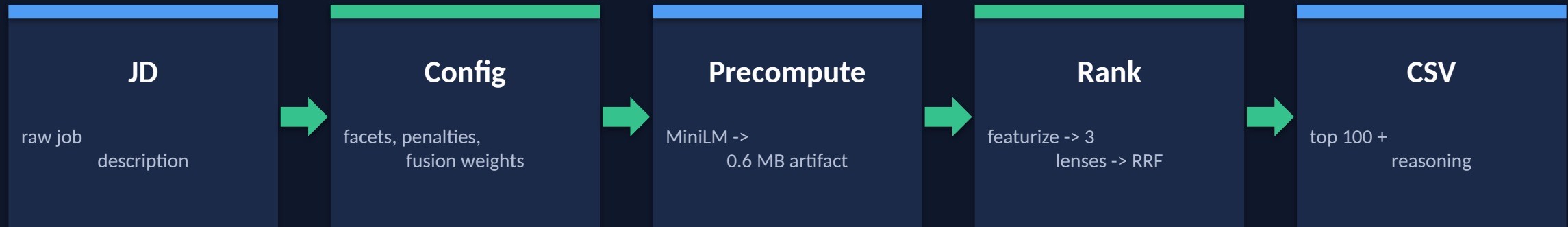
### 0 in top 100

Honeypots (impossible profiles: expert-but-0-months, tenure > experience, role longer than career) are detected and removed. Threshold for disqualification is >10%.

### High precision

We only flag logically-impossible profiles, not noisy heuristics, so legitimate candidates are never wrongly dropped.

## From JD to ranked CSV



**Offline boundary: only Precompute may use the network (one-time model download) and may exceed 5 min.**

The Rank step that produces the CSV is CPU-only, offline, and finishes in ~30s for the full 100k pool.

# Components and data flow

## INPUTS (offline)

### candidates.jsonl

100k profiles, each with 23 behavioral signals; managed via the web UI.

### per-job config

JD facets, disqualifier filters and fusion weights; one folder per JD.

### semantic\_scores.npz

0.6 MB MiniLM cosine-to-JD scores, precomputed once and committed.

## RANKING ENGINE (CPU, ~30s)

### featurize

one vectorized pandas/numpy pass: profile features + 23-signal availability modifier; flags impossible honeypots.

### 3 scorers

rules (JD-faithful) | TF-IDF (lexical) | MiniLM (semantic).

### RRF fusion

rank-based merge of the three scorers; honeypots excluded.

## OUTPUT + ACCESS

### top-100 CSV

candidate\_id, rank, 0-1 fit score, and evidence-based reasoning. Validator-clean, UTF-8.

### run it anywhere

rank.py one command | Docker (--network none) | Colab sandbox | FastAPI web UI to upload JDs, manage candidates, and run matches.

Data flow: profiles + JD config -> one vectorized featurization (+ honeypot flags) -> three scorers -> RRF -> honeypot-filtered top 100 with evidence.

# Quality and constraint compliance

**~30s**

full 100k ranked on CPU  
(Docker ~50s)

**0**

honeypots in the submitted top  
100

**0.6 MB**

committed semantic artifact  
(git-friendly)

**offline**

verified with docker --network  
none

## Constraint compliance

- ▶ Runtime  $\leq$  5 min: rank step ~30s for 100k
- ▶ Memory  $\leq$  16 GB: single vectorized pass
- ▶ CPU only / network off: no GPU, no API calls
- ▶ Disk  $\leq$  5 GB: artifact 0.6 MB, optional cache ~150 MB

## Confidence proxy (no ground truth)

- ▶ Scored on NDCG@10 (.50), NDCG@50 (.30), MAP (.15), P@10 (.05)
- ▶ Independent lenses cross-validate: a strong consensus core ranks in every strategy
- ▶ Agreement is reported by the ranker as a defensibility signal

# Drive the same engine from a browser

candidate → job · intelligent discovery & ranking

[Jobs](#)
[Candidates](#)
[Matching](#)

Pool

100,000

Run match...

Filter by job

All jobs

Filter by candidate set

All candidate sets

Runs (3)

| Job                                            | Candidate set           | Status | Pool | Top K | Took | Actions                                                                             |
|------------------------------------------------|-------------------------|--------|------|-------|------|-------------------------------------------------------------------------------------|
| Senior AI Engineer (Founding Team) - Redrob AI | Sample (first 50)       | done   | 50   | 100   | 9s   | <a href="#">View</a> <a href="#">CSV</a> <a href="#">PDF</a> <a href="#">Re-run</a> |
| Senior AI Engineer (Founding Team) - Redrob AI | Senior (8y+ experience) | done   | 400  | 100   | 6s   | <a href="#">View</a> <a href="#">CSV</a> <a href="#">PDF</a> <a href="#">Re-run</a> |
| Senior AI Engineer (Founding Team) - Redrob AI | AI / ML engineers       | done   | 400  | 100   | 4s   | <a href="#">View</a> <a href="#">CSV</a> <a href="#">PDF</a> <a href="#">Re-run</a> |

**Results**
Senior AI Engineer (Founding Team) - Redrob AI × Senior (8y+ experience) · 100 ranked · 400 in pool · took 6s

[Export CSV](#)
[Export PDF](#)
[Close](#)

| # | Candidate    | Score  | Reasoning                                                                                                                                                                                                                                                                                                                                                                                                   |
|---|--------------|--------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | CAND_0000883 | 1.0000 | Content Writer, 14.9y @ Wayne Enterprises, Wayne Enterprises. JD match -> vector DBs/hybrid search: FAISS(adv,5mo), Pinecone(int,6mo); embeddings retrieval: Embeddings(adv,13mo), Sentence Transformers(adv,11mo); ranking/recsys: Recommendation Systems(adv,16mo); LLM fine-tuning: Fine-tuning LLMs(adv,9mo). Signals: 9% recruiter response, active 2026-01, 120d notice. Concerns: inactive recently. |
| 2 | CAND_0000973 | 0.9837 | Software Engineer, 8y @ Globex Inc. JD match -> ranking/recsys: Recommendation Systems(adv,58mo); LLM fine-tuning: RAG(adv,22mo). Signals: 19% recruiter response, active 2026-03, 120d notice.                                                                                                                                                                                                             |
| 3 | CAND_0000930 | 0.9503 | Software Engineer, 9y @ Pied Piper, Zomato. JD match -> no direct JD-skill match (ranked on adjacent signals). Signals: 37% recruiter response, active 2026-04, 120d notice.                                                                                                                                                                                                                                |
| 4 | CAND_0000498 | 0.8995 | Content Writer, 10.7y @ Initech, Initech. JD match -> vector DBs/hybrid search: Pinecone(int,16mo); embeddings retrieval: Embeddings(adv,18mo), Sentence Transformers(adv,10mo); LLM fine-tuning: RAG(int,4mo). Signals: 37% recruiter response, active 2025-11, 60d notice. Concerns: inactive recently.                                                                                                   |
| 5 | CAND_0000133 | 0.8951 | Graphic Designer, 11.7y @ Acme Corp, Pied Piper. JD match -> vector DBs/hybrid search: Pinecone(adv,18mo); embeddings retrieval: Sentence Transformers(int,9mo), Semantic Search(adv,18mo); LLM fine-tuning: Fine-tuning LLMs(adv,5mo). Signals: 42% recruiter response, active 2026-01, 120d notice.                                                                                                       |
| 6 | CAND_0000553 | 0.8572 | Software Engineer, 9.3y @ Globex Inc, Razorpay. JD match -> LLM fine-tuning: RAG(int,33mo). Signals: 38% recruiter response, active 2026-05, 120d notice.                                                                                                                                                                                                                                                   |

► **Jobs & candidate sets:** default JD pre-loaded (markdown); curate candidate sets from the 100k pool.

► **Run-match popup:** tick job(s) x candidate set(s); queues every combination with live progress.

► **Filterable runs + paging:** filter runs by job or set; results paginate 100 per page.

► **Import / export:** CSV in (validated) and CSV or print-ready PDF out, in one click.

► **Same engine:** identical featurize -> 3 lenses -> RRF -> honeypot filter as rank.py.

100k pool ranked end-to-end, 0 honeypots in the top 100.

# And why each was chosen

## **pandas / numpy**

vectorized featurization and scoring; no per-row loops, so 100k fits the budget.

## **scikit-learn**

TF-IDF lexical recall, battle-tested and fast on CPU.

## **MiniLM (transformers)**

compact offline sentence embeddings for semantic fit, precomputed once.

## **Reciprocal Rank Fusion**

scale-free combination of heterogeneous lenses.

## **Docker**

exact Stage-3 parity: CPU-only, offline reproduction in one command.

## **Google Colab**

zero-setup hosted sandbox that runs end-to-end on a sample.

# Everything needed to reproduce and verify

- ▶ **GitHub:** [github.com/NavpreetDevpuri/canjob](https://github.com/NavpreetDevpuri/canjob) (public, real iterative history)
- ▶ **Reproduce:** `python rank.py --candidates ./candidates.jsonl --out ./submission.csv`
- ▶ **Sandbox:** Google Colab run-all notebook with a config panel and bundled sample
- ▶ **Docker:** `docker build -t canjob .` then `docker run --network none ...`
- ▶ **Web UI (bonus):** FastAPI + single-page app: upload JDs, manage candidates, run matches with live progress
- ▶ **Submission CSV:** top 100, UTF-8, `candidate_id,rank,score,reasoning` (validator-clean)
- ▶ **Metadata:** `submission_metadata.yaml` mirrors the portal fields
- ▶ **AI usage:** approach, architecture & decisions are the developer's; code written fast with Cursor; no LLM at rank time

# Thank you

---

**canjob** - Intelligent Candidate Discovery & Ranking

Navpreet Singh | [github.com/NavpreetDevpuri/canjob](https://github.com/NavpreetDevpuri/canjob)

Ideas & architecture by the developer; implemented quickly with Cursor (AI). No LLM/API at rank time.